

EventLoop



ReactPHP's core reactor event loop that libraries can use for evented I/O.

In order for async based libraries to be interoperable, they need to use the same event loop. This component provides a common **LoopInterface** that any library can target. This allows them to be used in the same loop, with one single **run()** call that is controlled by the user.

Table of contents

- Quickstart example
- Usage
 - Loop
 - * Loop methods
 - * Loop autorun
 - * get()
 - ~~Factory~~
 - * ~~create()~~
 - Loop implementations
 - * StreamSelectLoop
 - * ExtEventLoop
 - * ExtEvLoop
 - * ExtUvLoop
 - * ~~ExtLibeventLoop~~
 - * ~~ExtLibevLoop~~
 - LoopInterface
 - * run()
 - * stop()
 - * addTimer()
 - * addPeriodicTimer()
 - * cancelTimer()
 - * futureTick()
 - * addSignal()
 - * removeSignal()
 - * addReadStream()
 - * addWriteStream()
 - * removeReadStream()
 - * removeWriteStream()
- Install
- Tests
- License
- More

Quickstart example

Here is an async HTTP server built with just the event loop.

```
<?php

use React\EventLoop\Loop;

require __DIR__ . '/vendor/autoload.php';

$server = stream_socket_server('tcp://127.0.0.1:8080');
stream_set_blocking($server, false);

Loop::addReadStream($server, function ($server) {
    $conn = stream_socket_accept($server);
    $data = "HTTP/1.1 200 OK\r\nContent-Length: 3\r\n\r\nHi\n";
    Loop::addWriteStream($conn, function ($conn) use (&$data) {
        $written = fwrite($conn, $data);
        if ($written === strlen($data)) {
            fclose($conn);
            Loop::removeWriteStream($conn);
        } else {
            $data = substr($data, $written);
        }
    });
});

Loop::addPeriodicTimer(5, function () {
    $memory = memory_get_usage() / 1024;
    $formatted = number_format($memory, 3).'K';
    echo "Current memory usage: {$formatted}\n";
});
```

See also the examples.

Usage

Typical applications would use the Loop class to use the default event loop like this:

```
use React\EventLoop\Loop;

$timer = Loop::addPeriodicTimer(0.1, function () {
    echo 'Tick' . PHP_EOL;
});

Loop::addTimer(1.0, function () use ($timer) {
    Loop::cancelTimer($timer);
});
```

```
        echo 'Done' . PHP_EOL;
    });
```

As an alternative, you can also explicitly create an event loop instance at the beginning, reuse it throughout your program and finally run it at the end of the program like this:

```
$loop = React\EventLoop\Loop::get(); // or deprecated React\EventLoop\Factory::create();

$timer = $loop->addPeriodicTimer(0.1, function () {
    echo 'Tick' . PHP_EOL;
});

$loop->addTimer(1.0, function () use ($loop, $timer) {
    $loop->cancelTimer($timer);
    echo 'Done' . PHP_EOL;
});

$loop->run();
```

While the former is more concise, the latter is more explicit. In both cases, the program would perform the exact same steps.

1. The event loop instance is created at the beginning of the program. This is implicitly done the first time you call the `Loop` class or explicitly when using the deprecated `Factory::create()` method (or manually instantiating any of the loop implementations).
2. The event loop is used directly or passed as an instance to library and application code. In this example, a periodic timer is registered with the event loop which simply outputs `Tick` every fraction of a second until another timer stops the periodic timer after a second.
3. The event loop is run at the end of the program. This is automatically done when using the `Loop` class or explicitly with a single `run()` call at the end of the program.

As of v1.2.0, we highly recommend using the `Loop` class. The explicit loop instructions are still valid and may still be useful in some applications, especially for a transition period towards the more concise style.

Loop

The `Loop` class exists as a convenient global accessor for the event loop.

Loop methods The `Loop` class provides all methods that exist on the `LoopInterface` as static methods:

- `run()`
- `stop()`
- `addTimer()`

- addPeriodicTimer()
- cancelTimer()
- futureTick()
- addSignal()
- removeSignal()
- addReadStream()
- addWriteStream()
- removeReadStream()
- removeWriteStream()

If you're working with the event loop in your application code, it's often easiest to directly interface with the static methods defined on the `Loop` class like this:

```
use React\EventLoop\Loop;

$timer = Loop::addPeriodicTimer(0.1, function () {
    echo 'Tick' . PHP_EOL;
});

Loop::addTimer(1.0, function () use ($timer) {
    Loop::cancelTimer($timer);
    echo 'Done' . PHP_EOL;
});
```

On the other hand, if you're familiar with object-oriented programming (OOP) and dependency injection (DI), you may want to inject an event loop instance and invoke instance methods on the `LoopInterface` like this:

```
use React\EventLoop\Loop;
use React\EventLoop\LoopInterface;

class Greeter
{
    private $loop;

    public function __construct(LoopInterface $loop)
    {
        $this->loop = $loop;
    }

    public function greet(string $name)
    {
        $this->loop->addTimer(1.0, function () use ($name) {
            echo 'Hello ' . $name . '!' . PHP_EOL;
        });
    }
}
```

```
$greeter = new Greeter(Loop::get());
$greeter->greet('Alice');
$greeter->greet('Bob');
```

Each static method call will be forwarded as-is to the underlying event loop instance by using the `Loop::get()` call internally. See `LoopInterface` for more details about available methods.

Loop autorun When using the `Loop` class, it will automatically execute the loop at the end of the program. This means the following example will schedule a timer and will automatically execute the program until the timer event fires:

```
use React\EventLoop\Loop;

Loop::addTimer(1.0, function () {
    echo 'Hello' . PHP_EOL;
});
```

As of v1.2.0, we highly recommend using the `Loop` class this way and omitting any explicit `run()` calls. For BC reasons, the explicit `run()` method is still valid and may still be useful in some applications, especially for a transition period towards the more concise style.

If you don't want the `Loop` to run automatically, you can either explicitly `run()` or `stop()` it. This can be useful if you're using a global exception handler like this:

```
use React\EventLoop\Loop;

Loop::addTimer(10.0, function () {
    echo 'Never happens';
});

set_exception_handler(function (Throwable $e) {
    echo 'Error: ' . $e->getMessage() . PHP_EOL;
    Loop::stop();
});

throw new RuntimeException('Demo');
```

get() The `get(): LoopInterface` method can be used to get the currently active event loop instance.

This method will always return the same event loop instance throughout the lifetime of your application.

```
use React\EventLoop\Loop;
use React\EventLoop\LoopInterface;
```

```
$loop = Loop::get();
```

```
assert($loop instanceof LoopInterface);  
assert($loop === Loop::get());
```

This is particularly useful if you're using object-oriented programming (OOP) and dependency injection (DI). In this case, you may want to inject an event loop instance and invoke instance methods on the `LoopInterface` like this:

```
use React\EventLoop\Loop;  
use React\EventLoop\LoopInterface;  
  
class Greeter  
{  
    private $loop;  
  
    public function __construct(LoopInterface $loop)  
    {  
        $this->loop = $loop;  
    }  
  
    public function greet(string $name)  
    {  
        $this->loop->addTimer(1.0, function () use ($name) {  
            echo 'Hello ' . $name . '!' . PHP_EOL;  
        });  
    }  
}  
  
$greeter = new Greeter(Loop::get());  
$greeter->greet('Alice');  
$greeter->greet('Bob');
```

See `LoopInterface` for more details about available methods.

Factory

Deprecated since v1.2.0, see `Loop` class instead.

The deprecated `Factory` class exists as a convenient way to pick the best available event loop implementation.

create()

Deprecated since v1.2.0, see `Loop::get()` instead.

The deprecated `create(): LoopInterface` method can be used to create a new event loop instance:

```
// deprecated
$loop = React\EventLoop\Factory::create();

// new
$loop = React\EventLoop\Loop::get();
```

This method always returns an instance implementing `LoopInterface`, the actual event loop implementation is an implementation detail.

This method should usually only be called once at the beginning of the program.

Loop implementations

In addition to the `LoopInterface`, there are a number of event loop implementations provided.

All of the event loops support these features:

- File descriptor polling
- One-off timers
- Periodic timers
- Deferred execution on future loop tick

For most consumers of this package, the underlying event loop implementation is an implementation detail. You should use the `Loop` class to automatically create a new instance.

Advanced! If you explicitly need a certain event loop implementation, you can manually instantiate one of the following classes. Note that you may have to install the required PHP extensions for the respective event loop implementation first or they will throw a `BadMethodCallException` on creation.

StreamSelectLoop A `stream_select()` based event loop.

This uses the `stream_select()` function and is the only implementation that works out of the box with PHP.

This event loop works out of the box on PHP 5.3 through PHP 8+ and HHVM. This means that no installation is required and this library works on all platforms and supported PHP versions. Accordingly, the `Loop` class and the deprecated `Factory` will use this event loop by default if you do not install any of the event loop extensions listed below.

Under the hood, it does a simple `select` system call. This system call is limited to the maximum file descriptor number of `FD_SETSIZE` (platform dependent, commonly 1024) and scales with `O(m)` (`m` being the maximum file descriptor number passed). This means that you may run into issues when handling thousands of streams concurrently and you may want to look into using one of the alternative event loop implementations listed below in this case. If your use case is among the many common use cases that involve handling only dozens or

a few hundred streams at once, then this event loop implementation performs really well.

If you want to use signal handling (see also `addSignal()` below), this event loop implementation requires `ext-pcntl`. This extension is only available for Unix-like platforms and does not support Windows. It is commonly installed as part of many PHP distributions. If this extension is missing (or you're running on Windows), signal handling is not supported and throws a `BadMethodCallException` instead.

This event loop is known to rely on wall-clock time to schedule future timers when using any version before PHP 7.3, because a monotonic time source is only available as of PHP 7.3 (`hrtime()`). While this does not affect many common use cases, this is an important distinction for programs that rely on a high time precision or on systems that are subject to discontinuous time adjustments (time jumps). This means that if you schedule a timer to trigger in 30s on PHP < 7.3 and then adjust your system time forward by 20s, the timer may trigger in 10s. See also `addTimer()` for more details.

ExtEventLoop An `ext-event` based event loop.

This uses the `event` PECL extension, that provides an interface to `libevent` library. `libevent` itself supports a number of system-specific backends (epoll, kqueue).

This loop is known to work with PHP 5.4 through PHP 8+.

ExtEvLoop An `ext-ev` based event loop.

This loop uses the `ev` PECL extension, that provides an interface to `libev` library. `libev` itself supports a number of system-specific backends (epoll, kqueue).

This loop is known to work with PHP 5.4 through PHP 8+.

ExtUvLoop An `ext-uv` based event loop.

This loop uses the `uv` PECL extension, that provides an interface to `libuv` library. `libuv` itself supports a number of system-specific backends (epoll, kqueue).

This loop is known to work with PHP 7+.

ExtLibeventLoop

Deprecated since v1.2.0, use `ExtEventLoop` instead.

An `ext-libevent` based event loop.

This uses the `libevent` PECL extension, that provides an interface to `libevent` library. `libevent` itself supports a number of system-specific backends (epoll, kqueue).

This event loop does only work with PHP 5. An unofficial update for PHP 7 does exist, but it is known to cause regular crashes due to **SEGFaults**. To reiterate: Using this event loop on PHP 7 is not recommended. Accordingly, neither the **Loop** class nor the deprecated **Factory** class will try to use this event loop on PHP 7.

This event loop is known to trigger a readable listener only if the stream *becomes* readable (edge-triggered) and may not trigger if the stream has already been readable from the beginning. This also implies that a stream may not be recognized as readable when data is still left in PHP's internal stream buffers. As such, it's recommended to use `stream_set_read_buffer($stream, 0);` to disable PHP's internal read buffer in this case. See also `addReadStream()` for more details.

ExtLibevLoop

Deprecated since v1.2.0, use **ExtEvLoop** instead.

An **ext-libev** based event loop.

This uses an unofficial **libev** extension, that provides an interface to **libev** library. **libev** itself supports a number of system-specific backends (epoll, kqueue).

This loop does only work with PHP 5. An update for PHP 7 is unlikely to happen any time soon.

LoopInterface

run() The `run(): void` method can be used to run the event loop until there are no more tasks to perform.

For many applications, this method is the only directly visible invocation on the event loop. As a rule of thumb, it is usually recommended to attach everything to the same loop instance and then run the loop once at the bottom end of the application.

```
$loop->run();
```

This method will keep the loop running until there are no more tasks to perform. In other words: This method will block until the last timer, stream and/or signal has been removed.

Likewise, it is imperative to ensure the application actually invokes this method once. Adding listeners to the loop and missing to actually run it will result in the application exiting without actually waiting for any of the attached listeners.

This method **MUST NOT** be called while the loop is already running. This method **MAY** be called more than once after it has explicitly been `stop()`ped or after it automatically stopped because it previously did no longer have anything to do.

stop() The `stop(): void` method can be used to instruct a running event loop to stop.

This method is considered advanced usage and should be used with care. As a rule of thumb, it is usually recommended to let the loop stop only automatically when it no longer has anything to do.

This method can be used to explicitly instruct the event loop to stop:

```
$loop->addTimer(3.0, function () use ($loop) {
    $loop->stop();
});
```

Calling this method on a loop instance that is not currently running or on a loop instance that has already been stopped has no effect.

addTimer() The `addTimer(float $interval, callable $callback): TimerInterface` method can be used to enqueue a callback to be invoked once after the given interval.

The second parameter **MUST** be a timer callback function that accepts the timer instance as its only parameter. If you don't use the timer instance inside your timer callback function you **MAY** use a function which has no parameters at all.

The timer callback function **MUST NOT** throw an **Exception**. The return value of the timer callback function will be ignored and has no effect, so for performance reasons you're recommended to not return any excessive data structures.

This method returns a timer instance. The same timer instance will also be passed into the timer callback function as described above. You can invoke `cancelTimer` to cancel a pending timer. Unlike `addPeriodicTimer()`, this method will ensure the callback will be invoked only once after the given interval.

```
$loop->addTimer(0.8, function () {
    echo 'world!' . PHP_EOL;
});
```

```
$loop->addTimer(0.3, function () {
    echo 'hello ';
});
```

See also example #1.

If you want to access any variables within your callback function, you can bind arbitrary data to a callback closure like this:

```
function hello($name, LoopInterface $loop)
{
    $loop->addTimer(1.0, function () use ($name) {
        echo "hello $name\n";
    });
}
```

```
}
```

```
hello('Tester', $loop);
```

This interface does not enforce any particular timer resolution, so special care may have to be taken if you rely on very high precision with millisecond accuracy or below. Event loop implementations **SHOULD** work on a best effort basis and **SHOULD** provide at least millisecond accuracy unless otherwise noted. Many existing event loop implementations are known to provide microsecond accuracy, but it's generally not recommended to rely on this high precision.

Similarly, the execution order of timers scheduled to execute at the same time (within its possible accuracy) is not guaranteed.

This interface suggests that event loop implementations **SHOULD** use a monotonic time source if available. Given that a monotonic time source is only available as of PHP 7.3 by default, event loop implementations **MAY** fall back to using wall-clock time. While this does not affect many common use cases, this is an important distinction for programs that rely on a high time precision or on systems that are subject to discontinuous time adjustments (time jumps). This means that if you schedule a timer to trigger in 30s and then adjust your system time forward by 20s, the timer **SHOULD** still trigger in 30s. See also event loop implementations for more details.

addPeriodicTimer() The `addPeriodicTimer(float $interval, callable $callback): TimerInterface` method can be used to enqueue a callback to be invoked repeatedly after the given interval.

The second parameter **MUST** be a timer callback function that accepts the timer instance as its only parameter. If you don't use the timer instance inside your timer callback function you **MAY** use a function which has no parameters at all.

The timer callback function **MUST NOT** throw an **Exception**. The return value of the timer callback function will be ignored and has no effect, so for performance reasons you're recommended to not return any excessive data structures.

This method returns a timer instance. The same timer instance will also be passed into the timer callback function as described above. Unlike `addTimer()`, this method will ensure the callback will be invoked infinitely after the given interval or until you invoke `cancelTimer`.

```
$timer = $loop->addPeriodicTimer(0.1, function () {
    echo 'tick!' . PHP_EOL;
});

$loop->addTimer(1.0, function () use ($loop, $timer) {
    $loop->cancelTimer($timer);
    echo 'Done' . PHP_EOL;
});
```

See also example #2.

If you want to limit the number of executions, you can bind arbitrary data to a callback closure like this:

```
function hello($name, LoopInterface $loop)
{
    $n = 3;
    $loop->addPeriodicTimer(1.0, function ($timer) use ($name, $loop, &$n) {
        if ($n > 0) {
            --$n;
            echo "hello $name\n";
        } else {
            $loop->cancelTimer($timer);
        }
    });
}

hello('Tester', $loop);
```

This interface does not enforce any particular timer resolution, so special care may have to be taken if you rely on very high precision with millisecond accuracy or below. Event loop implementations **SHOULD** work on a best effort basis and **SHOULD** provide at least millisecond accuracy unless otherwise noted. Many existing event loop implementations are known to provide microsecond accuracy, but it's generally not recommended to rely on this high precision.

Similarly, the execution order of timers scheduled to execute at the same time (within its possible accuracy) is not guaranteed.

This interface suggests that event loop implementations **SHOULD** use a monotonic time source if available. Given that a monotonic time source is only available as of PHP 7.3 by default, event loop implementations **MAY** fall back to using wall-clock time. While this does not affect many common use cases, this is an important distinction for programs that rely on a high time precision or on systems that are subject to discontinuous time adjustments (time jumps). This means that if you schedule a timer to trigger in 30s and then adjust your system time forward by 20s, the timer **SHOULD** still trigger in 30s. See also event loop implementations for more details.

Additionally, periodic timers may be subject to timer drift due to re-scheduling after each invocation. As such, it's generally not recommended to rely on this for high precision intervals with millisecond accuracy or below.

cancelTimer() The `cancelTimer(TimerInterface $timer): void` method can be used to cancel a pending timer.

See also `addPeriodicTimer()` and example #2.

Calling this method on a timer instance that has not been added to this loop instance or on a timer that has already been cancelled has no effect.

futureTick() The `futureTick(callable $listener): void` method can be used to schedule a callback to be invoked on a future tick of the event loop.

This works very much similar to timers with an interval of zero seconds, but does not require the overhead of scheduling a timer queue.

The tick callback function **MUST** be able to accept zero parameters.

The tick callback function **MUST NOT** throw an **Exception**. The return value of the tick callback function will be ignored and has no effect, so for performance reasons you're recommended to not return any excessive data structures.

If you want to access any variables within your callback function, you can bind arbitrary data to a callback closure like this:

```
function hello($name, LoopInterface $loop)
{
    $loop->futureTick(function () use ($name) {
        echo "hello $name\n";
    });
}

hello('Tester', $loop);
```

Unlike timers, tick callbacks are guaranteed to be executed in the order they are enqueued. Also, once a callback is enqueued, there's no way to cancel this operation.

This is often used to break down bigger tasks into smaller steps (a form of cooperative multitasking).

```
$loop->futureTick(function () {
    echo 'b';
});
$loop->futureTick(function () {
    echo 'c';
});
echo 'a';
```

See also example #3.

addSignal() The `addSignal(int $signal, callable $listener): void` method can be used to register a listener to be notified when a signal has been caught by this process.

This is useful to catch user interrupt signals or shutdown signals from tools like supervisor or systemd.

The second parameter **MUST** be a listener callback function that accepts the signal as its only parameter. If you don't use the signal inside your listener callback function you **MAY** use a function which has no parameters at all.

The listener callback function **MUST NOT** throw an **Exception**. The return value of the listener callback function will be ignored and has no effect, so for performance reasons you're recommended to not return any excessive data structures.

```
$loop->addSignal(SIGINT, function (int $signal) {
    echo 'Caught user interrupt signal' . PHP_EOL;
});
```

See also example #4.

Signaling is only available on Unix-like platforms, Windows isn't supported due to operating system limitations. This method may throw a **BadMethodCallException** if signals aren't supported on this platform, for example when required extensions are missing.

Note: A listener can only be added once to the same signal, any attempts to add it more than once will be ignored.

removeSignal() The `removeSignal(int $signal, callable $listener): void` method can be used to remove a previously added signal listener.

```
$loop->removeSignal(SIGINT, $listener);
```

Any attempts to remove listeners that aren't registered will be ignored.

addReadStream()

Advanced! Note that this low-level API is considered advanced usage. Most use cases should probably use the higher-level readable Stream API instead.

The `addReadStream(resource $stream, callable $callback): void` method can be used to register a listener to be notified when a stream is ready to read.

The first parameter **MUST** be a valid stream resource that supports checking whether it is ready to read by this loop implementation. A single stream resource **MUST NOT** be added more than once. Instead, either call `removeReadStream()` first or react to this event with a single listener and then dispatch from this listener. This method **MAY** throw an **Exception** if the given resource type is not supported by this loop implementation.

The second parameter **MUST** be a listener callback function that accepts the stream resource as its only parameter. If you don't use the stream resource inside your listener callback function you **MAY** use a function which has no parameters at all.

The listener callback function **MUST NOT** throw an **Exception**. The return value of the listener callback function will be ignored and has no effect, so for performance reasons you're recommended to not return any excessive data structures.

If you want to access any variables within your callback function, you can bind arbitrary data to a callback closure like this:

```
$loop->addReadStream($stream, function ($stream) use ($name) {
    echo $name . ' said: ' . fread($stream);
});
```

See also example #11.

You can invoke `removeReadStream()` to remove the read event listener for this stream.

The execution order of listeners when multiple streams become ready at the same time is not guaranteed.

Some event loop implementations are known to only trigger the listener if the stream *becomes* readable (edge-triggered) and may not trigger if the stream has already been readable from the beginning. This also implies that a stream may not be recognized as readable when data is still left in PHP's internal stream buffers. As such, it's recommended to use `stream_set_read_buffer($stream, 0)`; to disable PHP's internal read buffer in this case.

addWriteStream()

Advanced! Note that this low-level API is considered advanced usage. Most use cases should probably use the higher-level writable Stream API instead.

The `addWriteStream(resource $stream, callable $callback): void` method can be used to register a listener to be notified when a stream is ready to write.

The first parameter **MUST** be a valid stream resource that supports checking whether it is ready to write by this loop implementation. A single stream resource **MUST NOT** be added more than once. Instead, either call `removeWriteStream()` first or react to this event with a single listener and then dispatch from this listener. This method **MAY** throw an **Exception** if the given resource type is not supported by this loop implementation.

The second parameter **MUST** be a listener callback function that accepts the stream resource as its only parameter. If you don't use the stream resource inside your listener callback function you **MAY** use a function which has no parameters at all.

The listener callback function **MUST NOT** throw an **Exception**. The return value of the listener callback function will be ignored and has no effect, so

for performance reasons you're recommended to not return any excessive data structures.

If you want to access any variables within your callback function, you can bind arbitrary data to a callback closure like this:

```
$loop->addWriteStream($stream, function ($stream) use ($name) {  
    fwrite($stream, 'Hello ' . $name);  
});
```

See also example #12.

You can invoke `removeWriteStream()` to remove the write event listener for this stream.

The execution order of listeners when multiple streams become ready at the same time is not guaranteed.

removeReadStream() The `removeReadStream(resource $stream): void` method can be used to remove the read event listener for the given stream.

Removing a stream from the loop that has already been removed or trying to remove a stream that was never added or is invalid has no effect.

removeWriteStream() The `removeWriteStream(resource $stream): void` method can be used to remove the write event listener for the given stream.

Removing a stream from the loop that has already been removed or trying to remove a stream that was never added or is invalid has no effect.

Install

The recommended way to install this library is through Composer. New to Composer?

This project follows SemVer. This will install the latest supported version:

```
composer require react/event-loop:^1.5
```

See also the CHANGELOG for details about version upgrades.

This project aims to run on any platform and thus does not require any PHP extensions and supports running on legacy PHP 5.3 through current PHP 8+ and HHVM. It's *highly recommended to use the latest supported PHP version* for this project.

Installing any of the event loop extensions is suggested, but entirely optional. See also event loop implementations for more details.

Tests

To run the test suite, you first need to clone this repo and then install all dependencies through Composer:

```
composer install
```

To run the test suite, go to the project root and run:

```
vendor/bin/phpunit
```

License

MIT, see LICENSE file.

More

- See our Stream component for more information on how streams are used in real-world applications.
- See our users wiki and the dependents on Packagist for a list of packages that use the EventLoop in real-world applications.